You should see some output showing the Rails server starting up, similar to:

```
web_1  | => Booting Puma
web_1  | => Rails 5.2.2 application starting in development
web_1  | => Run `rails server -h` for more startup options
web_1  | Puma starting in single mode...
web_1  | * Version 3.12.0 (ruby 2.6.0-p0), codename: Llamas in Pajamas
web_1  | * Min threads: 5, max threads: 5
web_1  | * Environment: development
web_1  | * Listening on tcp://0.0.0.0:3000
web_1  | Use Ctrl-C to stop
```

The `-f` flag tells the command to follow the output—that is, to stay connected and continue to append to the screen any new output to the logs, similar to the `tail` Unix command.

Press `Ctrl`-`C` to terminate the logging stream.

It's important to realize this command displays the *container output* logs rather than the Rails server logs, which, by default, are stored in the `log/` directory. However, as we'll see later, in Part II, Toward Production, on page 127, it's common to configure Rails to log to `stdout` when using Docker.

Check out Docker's documentation[4] for more `docker logs` options.

Running One-Off Commands

Up until now, we've only run our `web` container using the image's default `CMD`, which starts a Rails server. What if we need to run a different command? For example, frequently we'll want to do things like migrate our database with `db:migrate`, run our tests, update our gems, or perform the many Rails commands we're used to running as part of our development. How do we do that?

There are actually two different ways to achieve this, which we'll demonstrate with a trivial example: `echo`ing something to the screen. Don't let this trivial example fool you; the approaches use the same mechanism we'll employ to run all our favorite commands in the upcoming chapters.

First, we can use `docker run` to start a new container for our one-off command. We provide the command after the service name, as follows, which overrides any default commands specified either in the `docker-compose.yml` file or in the `Dockerfile` itself:

```
$ docker-compose run --rm web echo 'ran a different command'
ran a different command
```

4. https://docs.docker.com/engine/reference/commandline/logs/